

STA Concepts

This chapter describes the basics of CMOS technology and the terminology involved in performing static timing analysis.

2.1 CMOS Logic Design

2.1.1 Basic MOS Structure

The physical implementation of MOS transistors (NMOS¹ and PMOS²) is depicted in Figure 2-1. The separation between the *source* and *drain* regions is the *length* of the MOS transistor. The smallest length used to build a MOS

-
1. N-channel Metal Oxide Semiconductor.
 2. P-channel Metal Oxide Semiconductor.

transistor is normally the smallest feature size for the CMOS technology process. For example, a $0.25\mu\text{m}$ technology allows MOS transistors with a channel length of $0.25\mu\text{m}$ or larger to be fabricated. By shrinking the channel geometry, the transistor size becomes smaller, and subsequently more transistors can be packed in a given area. As we shall see later in this chapter, this also allows the designs to operate at a greater speed.

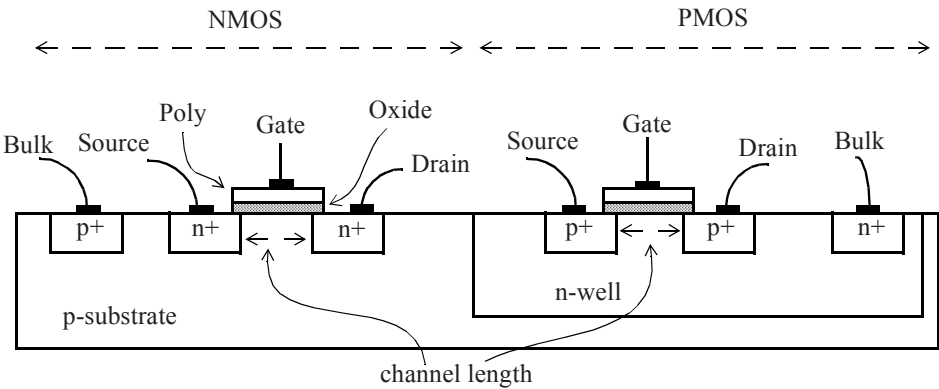


Figure 2-1 Structure of NMOS and PMOS transistors.

2.1.2 CMOS Logic Gate

A CMOS logic gate is built using NMOS and PMOS transistors. Figure 2-2 shows an example of a CMOS inverter. There are two stable states of the CMOS inverter depending upon the state of the input. When input A is low (at V_{ss} or logic-0), the NMOS transistor is *off* and the PMOS transistor is *on*, causing the output Z to be pulled to V_{dd} , which is a logic-1. When input A is high (at V_{dd} or logic-1), the NMOS transistor is *on* and the PMOS transistor is *off*, causing the output Z to be pulled to V_{ss} , which is a logic-0. In either of the two states described above, the CMOS inverter is stable and

$V_{ss} = 0$

does not draw any current¹ from the input A or from the power supply V_{dd} .

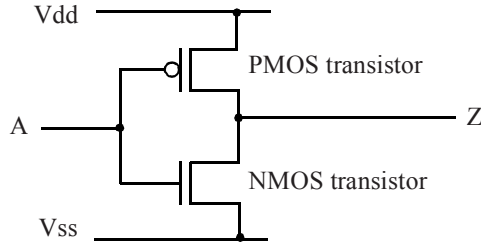


Figure 2-2 A CMOS inverter.

The characteristics of the CMOS inverter can be extended to any CMOS logic gate. In a CMOS logic gate, the output node is connected by a pull-up structure (made up of PMOS transistors) to V_{dd} and a pull-down structure (made up of NMOS transistors) to V_{ss} . As an example, a two-input CMOS *nand* gate is shown in Figure 2-3. In this example, the pull-up structure is comprised of the two parallel PMOS transistors and the pull-down structure is made up of two series NMOS transistors.

For any CMOS logic gate, the pull-up and pull-down structures are complementary. For inputs at logic-0 or logic-1, this means that if the pull-up stage is turned *on*, the pull-down stage will be *off* and similarly if the pull-up stage is turned *off*, the pull-down stage will be turned *on*. The pull-down and pull-up structures are governed by the logic function implemented by the CMOS gate. For example, in a CMOS *nand* gate, the function controlling the pull-down structure is "*A and B*", that is, the pull-down is turned *on* when A and B are both at logic-1. Similarly, the function controlling the pull-up structure is "*not A or not B*", that is, the pull-up is turned *on* when either A or B is at logic-0. These characteristics ensure that the output node logic will be pulled to V_{dd} based upon the function controlling the pull-up structure. Since the pull-down structure is controlled by a comple-

1. Depending upon the specifics of the CMOS technology, there is a small amount of leakage current that is drawn even in steady state.

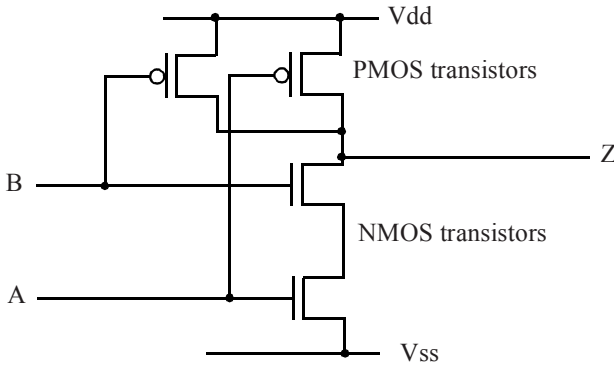


Figure 2-3 CMOS two-input NAND gate.

mentary function, the output node is at logic-0 when the pull-up structure function evaluates to 0.

For inputs at logic-0 or at logic-1, the CMOS logic gate does not draw any current from the inputs or from the power supply in steady state since the pull-up and pull-down structures can not both be *on*¹. Another important aspect of CMOS logic is that the inputs pose only a capacitive load to the previous stage.

The CMOS logic gate is an inverting gate which means that a single switching input (rising or falling) can only cause the output to switch in the opposite direction, that is, the output can not switch in the same direction as the switching input. The CMOS logic gates can however be cascaded to put together a more complex logic function - inverting as well as non-inverting.

2.1.3 Standard Cells

Most of the complex functionality in a chip is normally designed using basic building blocks which implement simple logic functions such as *and*, *or*, *nand*, *nor*, *and-or-invert*, *or-and-invert* and *flip-flop*. These basic building

1. The pull-up and pull-down structures are both *on* only during switching.

blocks are pre-designed and referred to as **standard cells**. The functionality and timing of the standard cells is pre-characterized and available to the designer. The designer can then implement the required functionality using the standard cells as the building blocks.

The key characteristics of the CMOS logic gates described in previous subsection are applicable to all CMOS digital designs. All digital CMOS cells are designed such that there is no current drawn from power supply (except for leakage) when the inputs are in a stable logic state. Thus, most of the power dissipation is related to the activity in the design and is caused by the charging and discharging of the inputs of CMOS cells in the design.

What is a logic-1 or a logic-0? In a CMOS cell, two values V_{IHmin} and V_{ILmax} define the limits. That is, any voltage value above V_{IHmin} is considered as a **logic-1** and any voltage value below V_{ILmax} is considered as a **logic-0**. See Figure 2-4. Typical values for a CMOS 0.13 μ m inverter cell with 1.2V V_{dd} supply are 0.465V for V_{ILmax} and 0.625V for V_{IHmin} . The V_{IHmin} and V_{ILmax} values are derived from the DC transfer characteristics of the cell. The DC transfer characteristics are described in greater detail in Section 6.2.3.

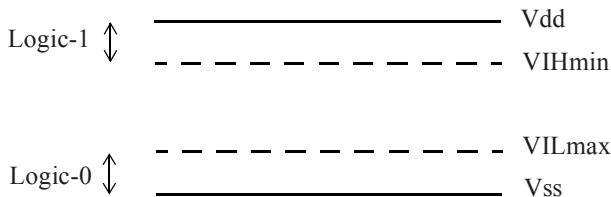


Figure 2-4 CMOS logic levels.

For more details on CMOS technology, refer to one of the relevant texts listed in the Bibliography.

2.2 Modeling of CMOS Cells

If a cell output pin drives multiple fanout cells, the total capacitance on the output pin of the cell is the sum of all the input capacitances of the cells that it is driving plus the sum of the capacitance of all the wire segments that comprise the net plus the output capacitance of the driving cell. Note that in a CMOS cell, the inputs to the cell present a capacitive load only.

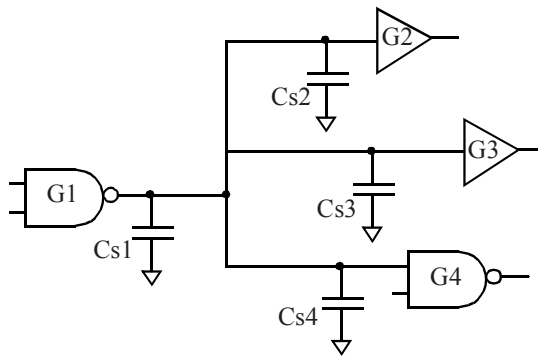


Figure 2-5 *Capacitance on a net.*

Figure 2-5 shows an example of a cell $G1$ driving three other cells $G2$, $G3$, and $G4$. $Cs1$, $Cs2$, $Cs3$ and $Cs4$ are the capacitance values of wire segments that comprise the net. Thus:

```
Total cap (Output  $G1$ ) = Cout( $G1$ ) + Cin( $G2$ ) + Cin( $G3$ ) +
                          Cin( $G4$ ) + Cs1 + Cs2 + Cs3 + Cs4
# Cout is the output pin capacitance of the cell.
# Cin is the input pin capacitance of the cell.
```

This is the capacitance that needs to be charged or discharged when cell $G1$ switches and thus this total capacitance value impacts the timing of cell $G1$.

From a timing perspective, we need to model the CMOS cell to aid us in analyzing the timing through the cell. An input pin capacitance is specified

for each of the input pins. There can also be an output pin capacitance though most CMOS logic cells do not include the pin capacitance for the output pins.

When output is a logic-1, the pull-up structure for the output stage is *on*, and it provides a path from the output to V_{dd} . Similarly, when the output is a logic-0, the pull-down structure for the output stage provides a path from the output to V_{ss} . When the CMOS cell switches state, the speed of the switching is governed by how fast the capacitance on the output net can be charged or discharged. The capacitance on the output net (Figure 2-5) is charged and discharged through the pull-up and pull-down structures respectively. Note that the channel in the pull-up and pull-down structures poses resistances for the output charging and discharging paths. The charging and discharging path resistances are a major factor in determining the speed of the CMOS cell. The inverse of the pull-up resistance is called the **output high drive** of the cell. The larger the output pull-up structure, the smaller the pull-up resistance and the larger the output high drive of the cell. The larger output structures also mean that the cell is larger in area. The smaller the output pull-up structures, the cell is smaller in area, and its output high drive is also smaller. The same concept for the pull-up structure can be applied for the pull-down structure which determines the resistance of the pull-down path and **output low drive**. In general, the cells are designed to have similar drive strengths (both large or both small) for pull-up and pull-down structures.

The output drive determines the maximum capacitive load that can be driven. The maximum capacitive load determines the maximum number of fanouts, that is, how many other cells it can drive. A higher output drive corresponds to a lower output pull-up and pull-down resistance which allows the cell to charge and discharge a higher load at the output pin.

Figure 2-6 shows an equivalent abstract model for a CMOS cell. The objective of this model is to abstract the timing behavior of the cell, and thus only the input and output stages are modeled. This model does not capture the intrinsic delay or the electrical behavior of the cell.

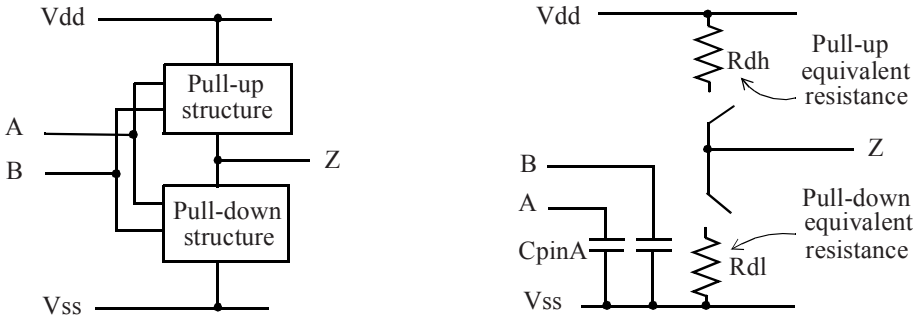


Figure 2-6 CMOS cell and its electrically equivalent model.

C_{pinA} is the input pin capacitance of the cell on input A. R_{dh} and R_{dl} are the output drive resistances of the cell and determine the rise and fall times of the output pin Z based upon the load being driven by the cell. This drive also determines the maximum fanout limit of the cell.

Figure 2-7 shows the same net as in Figure 2-5 but with the equivalent models for the cells.

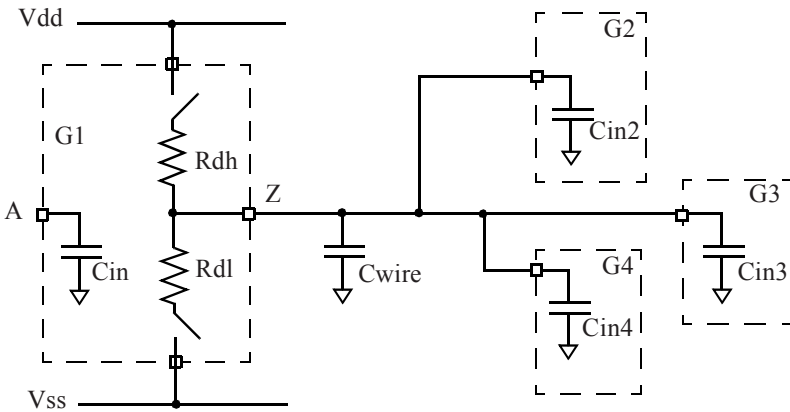


Figure 2-7 Net with CMOS equivalent models.

$$\begin{aligned}
 C_{\text{wire}} &= C_{s1} + C_{s2} + C_{s3} + C_{s4} \\
 \text{Output charging delay (for high or low)} &= \\
 R_{\text{out}} * (C_{\text{wire}} + C_{\text{in2}} + C_{\text{in3}} + C_{\text{in4}})
 \end{aligned}$$

In the above expression, R_{out} is one of R_{dh} or R_{dl} where R_{dh} is the output drive resistance for pull-up and R_{dl} is the output drive resistance for pull-down.

2.3 Switching Waveform

When a voltage is applied to the RC network as shown in Figure 2-8(a) by activating the $SW0$ switch, the output goes to a logic-1. Assuming the output is at 0V when $SW0$ is activated, the voltage transition at the output is described by the equation:

$$V = V_{\text{dd}} * [1 - e^{-t/(R_{\text{dh}} * C_{\text{load}})}]$$

The voltage waveform for this rise is shown in Figure 2-8(b). The product $(R_{\text{dh}} * C_{\text{load}})$ is called the **RC time constant** - typically this is also related to the transition time of the output.

When the output goes from logic-1 to logic-0, caused by input changes disconnecting $SW0$ and activating $SW1$, the output transition looks like the one shown in Figure 2-8(c). The output capacitance discharges through the $SW1$ switch which is *on*. The voltage transition in this case is described by the equation:

$$V = V_{\text{dd}} * e^{-t/(R_{\text{dl}} * C_{\text{load}})}$$

In a CMOS cell, the output charging and discharging waveforms do not appear like the RC charging and discharging waveforms of Figure 2-8 since the PMOS pull-up and the NMOS pull-down transistors are both *on* simultaneously for a brief amount of time. Figure 2-9 shows the *current* paths within a CMOS inverter cell for various stages of output switching from logic-1 to logic-0. Figure 2-9(a) shows the *current* flow when both the pull-

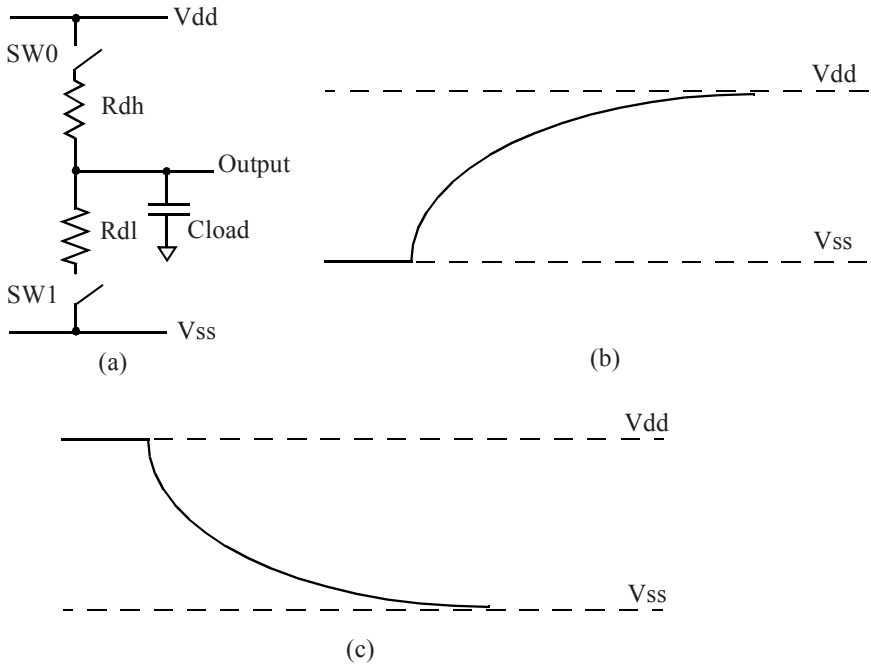
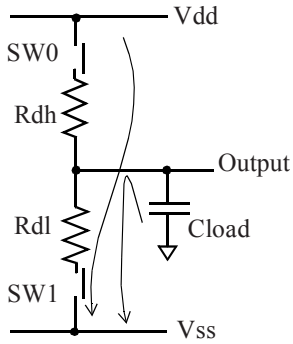


Figure 2-8 *RC charging and discharging waveforms.*

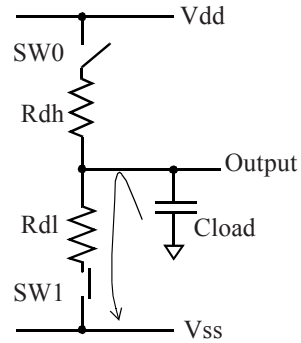
up and pull-down structures are *on*. Later, the pull-up structure turns *off* and the current flow is depicted in Figure 2-9(b). After the output reaches the final state, there is no current flow as the capacitance C_{load} is completely discharged.

Figure 2-10(a) shows a representative waveform at the output of a CMOS cell. Notice how the transition waveforms curve asymptotically towards the V_{ss} rail and the V_{dd} rail, and the linear portion of the waveform is in the middle.

In this text, we shall depict some waveforms using a simplistic drawing as shown in Figure 2-10(b). It shows the waveform with some transition time, which is the time needed to transition from one logic state to the other. Fig-



(a) Cell is switching.
(Pull-up, pull-down both *on*)



(b) Cell is discharging to logic-0.
(Pull-up *off*, pull-down *on*)

Figure 2-9 Current flow for a CMOS cell output stage.

Figure 2-10(c) shows the same waveforms using a transition time of 0, that is, as ideal waveforms. We shall be using both of these forms in this text interchangeably to explain the concepts, though in reality, each waveform has its real edge characteristics as shown in Figure 2-10(a).

2.4 Propagation Delay

Consider a CMOS inverter cell and its input and output waveforms. The **propagation delay** of the cell is defined with respect to some measurement points on the switching waveforms. Such points are defined using the following four variables:

```
# Threshold point of an input falling edge:
input_threshold_pct_fall : 50.0;
# Threshold point of an input rising edge:
input_threshold_pct_rise : 50.0;
# Threshold point of an output falling edge:
output_threshold_pct_fall : 50.0;
```

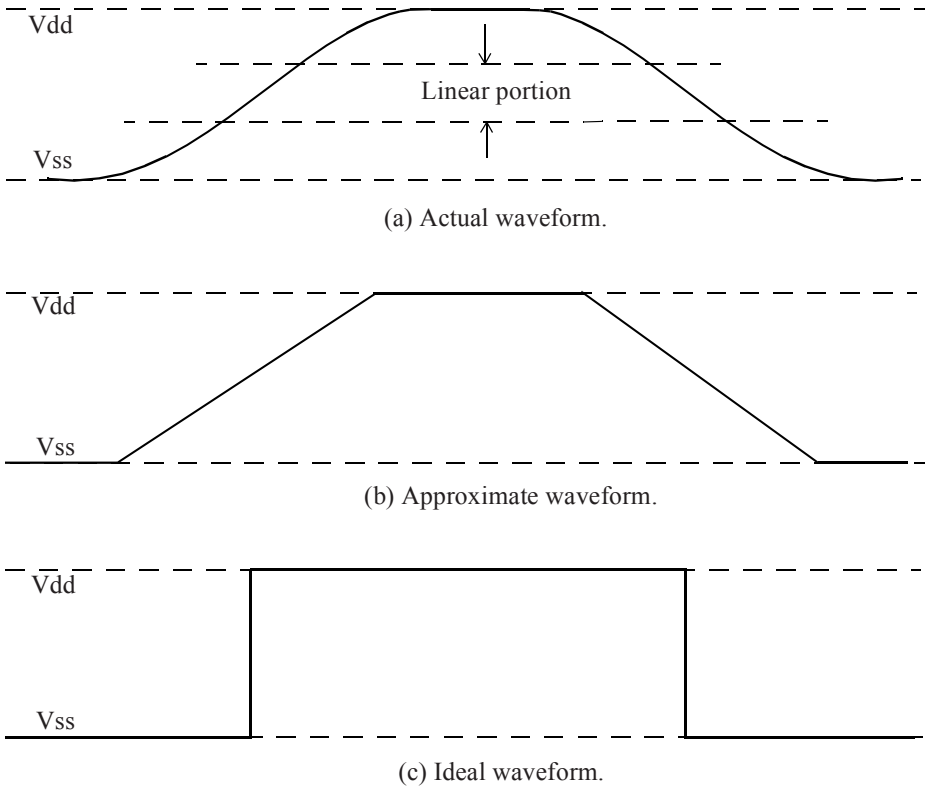


Figure 2-10 *CMOS output waveforms.*

```
# Threshold point of an output rising edge:  
output_threshold_pct_rise : 50.0;
```

These variables are part of a command set used to describe a cell library (this command set is described in Liberty¹). These threshold specifications are in terms of the percent of V_{dd} , or the power supply. Typically 50% threshold is used for delay measurement for most standard cell libraries.

1. See [LIB] in Bibliography.

Rising edge is the transition from logic-0 to logic-1. Falling edge is the transition from logic-1 to logic-0.

Consider the example inverter cell and the waveforms at its pins shown in Figure 2-11. The propagation delays are represented as:

- i. Output fall delay (T_f)
- ii. Output rise delay (T_r)

In general, these two values are different. Figure 2-11 shows how these two propagation delays are measured.

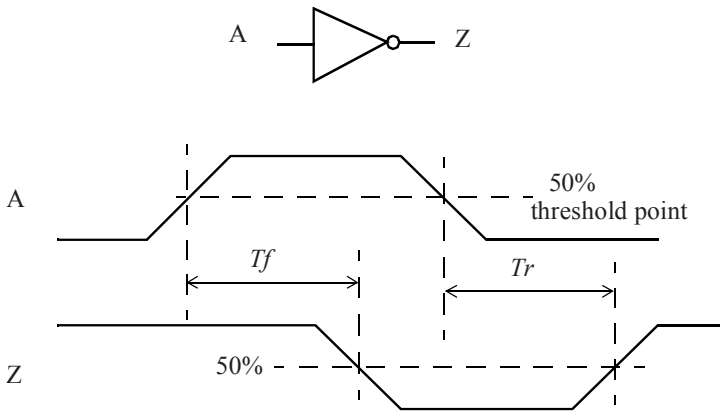


Figure 2-11 *Propagation delays.*

If we were looking at ideal waveforms, propagation delay would simply be the delay between the two edges. This is shown in Figure 2-12.

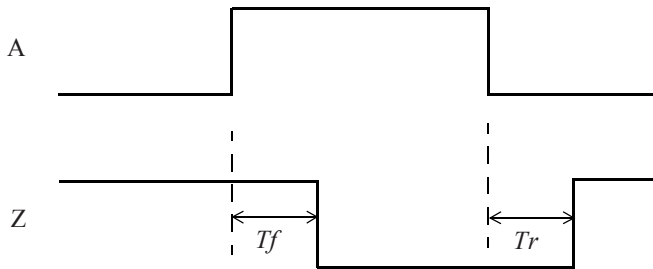


Figure 2-12 *Propagation delay using ideal waveforms.*

2.5 Slew of a Waveform

A slew rate is defined as a rate of change. In static timing analysis, the rising or falling waveforms are measured in terms of whether the transition is slow or fast. The slew is typically measured in terms of the **transition time**, that is, the time it takes for a signal to transition between two specific levels. Note that the transition time is actually inverse of the slew rate - the larger the transition time, the slower the slew, and vice versa. Figure 2-10 illustrates a typical waveform at the output of a CMOS cell. The waveforms at the ends are asymptotic and it is hard to determine the exact start and end points of the transition. Consequently, the transition time is defined with respect to specific threshold levels. For example, the slew threshold settings can be:

```
# Falling edge thresholds:
slew_lower_threshold_pct_fall : 30.0;
slew_upper_threshold_pct_fall : 70.0;
# Rising edge thresholds:
slew_lower_threshold_pct_rise : 30.0;
slew_upper_threshold_pct_rise : 70.0;
```

These values are specified as a percent of V_{dd} . The threshold settings specify that falling slew is the difference between the times that falling edge

reaches 70% and 30% of V_{dd} . Similarly, the settings for rise specify that the rise slew is the difference in times that the rising edge reaches 30% and 70% of V_{dd} . Figure 2-13 shows this pictorially.

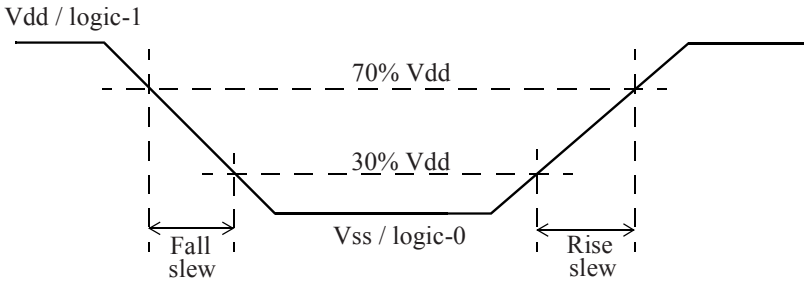


Figure 2-13 *Rise and fall transition times.*

Figure 2-14 shows another example where the slew on a falling edge is measured 20-80 (80% to 20%) and that on the rising edge is measured 10-90 (10% to 90%). Here are the threshold settings for this case.

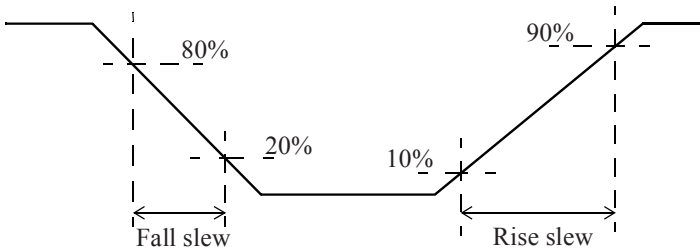


Figure 2-14 *Another example of slew measurement.*

```
# Falling edge thresholds:
slew_lower_threshold_pct_fall : 20.0;
slew_upper_threshold_pct_fall : 80.0;
```

```
# Rising edge thresholds:
slew_lower_threshold_pct_rise : 10.0;
slew_upper_threshold_pct_rise : 90.0;
```

2.6 Skew between Signals

Skew is the difference in timing between two or more signals, maybe data, clock or both. For example, if a clock tree has 500 end points and has a skew of 50ps, it means that the difference in latency between the longest path and the shortest clock path is 50ps. Figure 2-15 shows an example of a clock tree. The beginning point of a clock tree typically is a node where a clock is defined. The end points of a clock tree are typically clock pins of synchronous elements, such as flip-flops. **Clock latency** is the total time it takes from the clock source to an end point. **Clock skew** is the difference in arrival times at the end points of the clock tree.

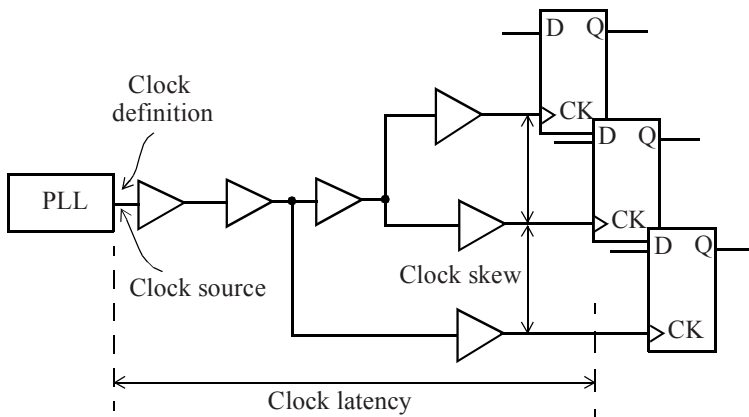


Figure 2-15 Clock tree, clock latency and clock skew.

An **ideal clock tree** is one where the clock source is assumed to have an infinite drive, that is, the clock can drive infinite sources with no delay. In addition, any cells present in the clock tree are assumed to have zero delay. In

the early stages of logical design, STA is often performed with ideal clock trees so that the focus of the analysis is on the data paths. In an ideal clock tree, clock skew is 0ps by default. Latency of a clock tree can be explicitly specified using the **set_clock_latency** command. The following example models the latency of a clock tree:

```
set_clock_latency 2.2 [get_clocks BZCLK]  
# Both rise and fall latency is 2.2ns.  
# Use options -rise and -fall if different.
```

Clock skew for a clock tree can also be implied by explicitly specifying its value using the **set_clock_uncertainty** command:

```
set_clock_uncertainty 0.250 -setup [get_clocks BZCLK]  
set_clock_uncertainty 0.100 -hold [get_clocks BZCLK]
```

The *set_clock_uncertainty* specifies a window within which a clock edge can occur. The uncertainty in the timing of the clock edge is to account for several factors such as clock period jitter and additional margins used for timing verification. Every real clock source has a finite amount of jitter - a window within which a clock edge can occur. The clock period jitter is determined by the type of clock generator utilized. In reality, there are no ideal clocks, that is, all clocks have a finite amount of jitter and the clock period jitter should be included while specifying the clock uncertainty.

Before the clock tree is implemented, the clock uncertainty must also include the expected clock skew of the implementation.

One can specify different clock uncertainties for setup checks and for hold checks. The hold checks do not require the clock jitter to be included in the uncertainty and thus a smaller value of clock uncertainty is generally specified for hold.

Figure 2-16 shows an example of a clock with a setup uncertainty of 250ps. Figure 2-16(b) shows how the uncertainty takes away from the time avail-

able for the logic to propagate to the next flip-flop stage. This is equivalent to validating the design to run at a higher frequency.

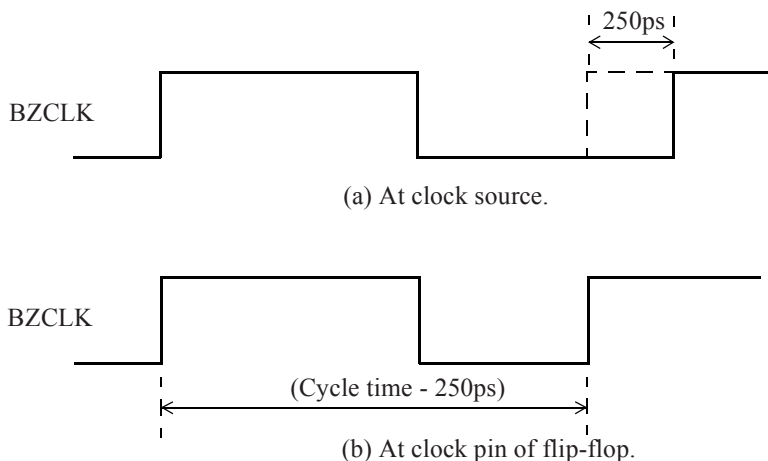


Figure 2-16 *Clock setup uncertainty.*

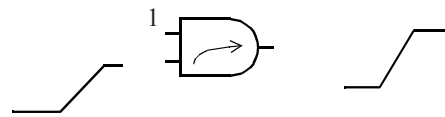
As specified above, the `set_clock_uncertainty` can also be used to model any additional margin. For example, a designer may use a 50ps timing margin as additional pessimism during design. This component can be added and included in the `set_clock_uncertainty` command. In general, before the clock tree is implemented, the `set_clock_uncertainty` command is used to specify a value that includes clock jitter plus estimated clock skew plus additional pessimism.

```
set_clock_latency 2.0 [get_clocks USBCLK]
set_clock_uncertainty 0.2 [get_clocks USBCLK]
# The 200ps may be composed of 50ps clock jitter,
# 100ps clock skew and 50ps additional pessimism.
```

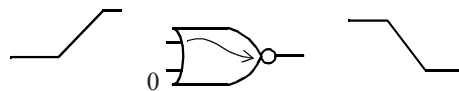
We shall see later how `set_clock_uncertainty` influences setup and hold checks. It is best to think of clock uncertainty as an offset to the final slack calculation.

2.7 Timing Arcs and Unateness

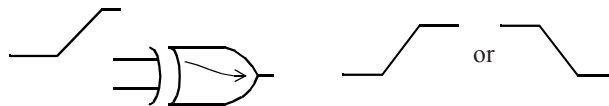
Every cell has multiple timing arcs. For example, a combinational logic cell, such as *and*, *or*, *nand*, *nor*, *adder* cell, has timing arcs from each input to each output of the cell. Sequential cells such as flip-flops have timing arcs from the clock to the outputs and timing constraints for the data pins with respect to the clock. Each timing arc has a timing sense, that is, how the output changes for different types of transitions on input. The timing arc is **positive unate** if a rising transition on an input causes the output to rise (or not to change) and a falling transition on an input causes the output to fall (or not to change). For example, the timing arcs for *and* and *or* type cells are positive unate. See Figure 2-17(a).



(a) Positive unate arc.



(b) Negative unate arc.



(c) Non-unate arc.

Figure 2-17 *Timing sense of arcs.*

A **negative unate** timing arc is one where a rising transition on an input causes the output to have a falling transition (or not to change) and a fall-

ing transition on an input causes the output to have a rising transition (or not to change). For example, the timing arcs for *nand* and *nor* type cells are negative unate. See Figure 2-17(b).

In a **non-unate** timing arc, the output transition cannot be determined solely from the direction of change of an input but also depends upon the state of the other inputs. For example, the timing arcs in an *xor* cell (exclusive-or) are non-unate.¹ See Figure 2-17(c).

Unateness is important for timing as it specifies how the edges (transitions) can propagate through a cell and how they appear at the output of the cell.

One can take advantage of the non-unateness property of a timing arc, such as when an *xor* cell is used, to invert the polarity of a clock. See the example in Figure 2-18. If input *POLCTRL* is a logic-0, the clock *DDRCLK* on output of the cell *UXOR0* has the same polarity as the input clock *MEMCLK*. If *POLCTRL* is a logic-1, the clock on the output of the cell *UXOR0* has the opposite polarity as the input clock *MEMCLK*.

2.8 Min and Max Timing Paths

The total delay for the logic to propagate through a logic path is referred to as the **path delay**. This corresponds to the sum of the delays through the various logic cells and nets along the path. In general, there are multiple paths through which the logic can propagate to the required destination point. The actual path taken depends upon the state of the other inputs along the logic path. An example is illustrated in Figure 2-19. Since there are multiple paths to the destination, the maximum and minimum timing to the destination points can be obtained. The paths corresponding to the maximum timing and minimum timing are referred to as the max path and min path respectively. A **max path** between two end points is the path with the largest delay (also referred to as the **longest path**). Similarly, a **min**

1. It is possible to specify state-dependent timing arcs for an *xor* cell which are positive unate and negative unate. This is described in Chapter 3.

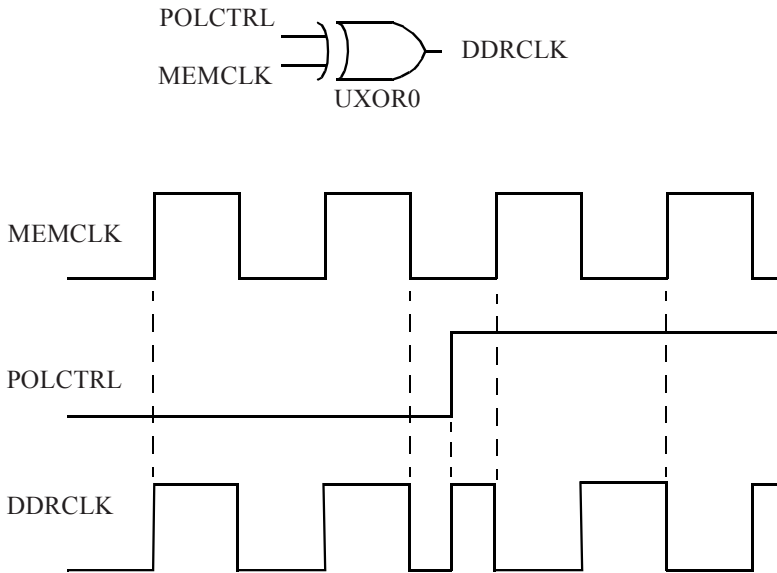


Figure 2-18 *Controlling clock polarity using non-unate cell.*

path is the path with the smallest delay (also referred to as the **shortest path**). Note that the longest and shortest refer to the cumulative delay of the path, not to the number of cells in the path.

Figure 2-19 shows an example of a data path between flip-flops. A max path between flip-flops *UFF1* and *UFF3* is assumed to be the one that goes through *UNAND0*, *UBUF2*, *UOR2* and *UNAND6* cells. A min path between the flip-flops *UFF1* and *UFF3* is assumed to be the one that goes through the *UOR4* and *UNAND6* cells. Note that in this example, the max and min are with reference to the destination point which is the *D* pin of the flip-flop *UFF3*.

A max path is often called a **late path**, while a min path is often called an **early path**.

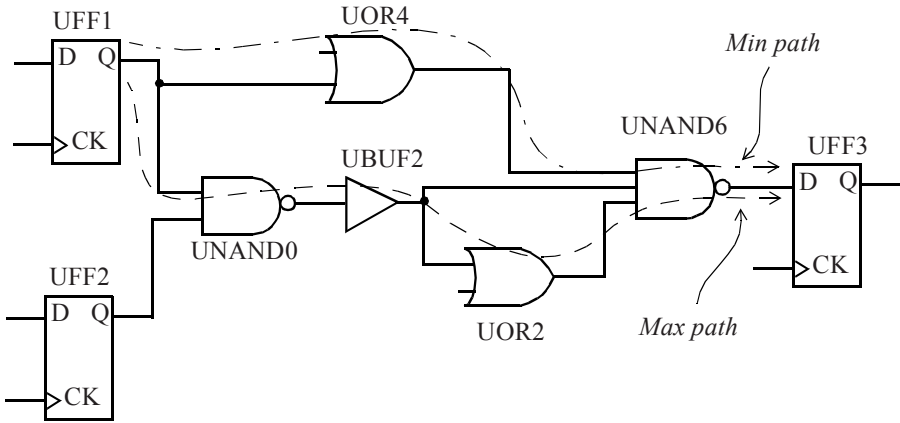


Figure 2-19 *Max and min timing paths.*

When a flip-flop to flip-flop path such as from *UFF1* to *UFF3* is considered, one of the flip-flops launches the data and the other flip-flop captures the data. In this case, since *UFF1* launches the data, *UFF1* is referred to as the **launch** flip-flop. And since *UFF3* captures the data, *UFF3* is referred to as the **capture** flip-flop. Notice that the launch and capture terminology are always with reference to a flip-flop to flip-flop path. For example, *UFF3* would become a launch flip-flop for the path to whatever flip-flop captures the data produced by *UFF3*.

2.9 Clock Domains

In synchronous logic design, a periodic clock signal latches the new data computed into the flip-flops. The new data inputs are based upon the flip-flop values from a previous clock cycle. The latched data thus gets used for computing the data for the next clock cycle.

A clock typically feeds a number of flip-flops. The set of flip-flops being fed by one clock is called its **clock domain**. In a typical design, there may be

more than one clock domain. For example, 200 flip-flops may be clocked by *USBCLK* and 1000 flip-flops may be fed by clock *MEMCLK*. Figure 2-20 depicts the flip-flops along with the clocks. In this example, we say that there are two clock domains.

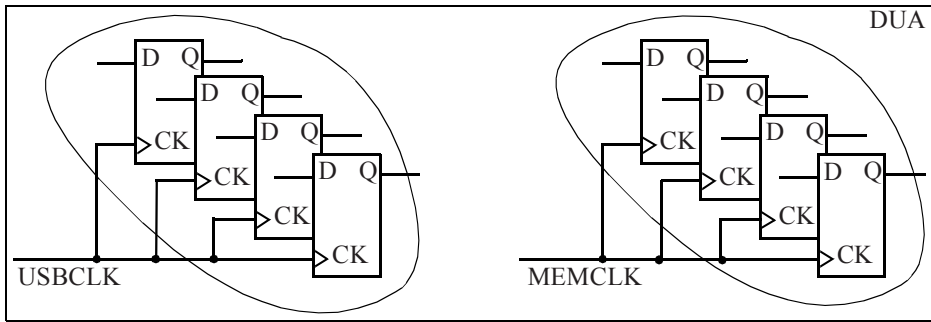


Figure 2-20 *Two clock domains.*

A question of interest is whether the clock domains are related or independent of each other. The answer depends on whether there are any data paths that start from one clock domain and end in the other clock domain. If there are no such paths, we can safely say that the two clock domains are independent of each other. This means that there is no timing path that starts from one clock domain and ends in the other clock domain.

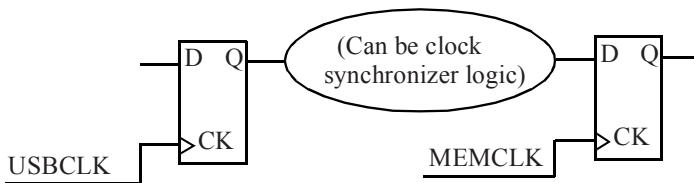


Figure 2-21 *Clock domain crossing.*

If indeed there are data paths that cross between clock domains (see Figure 2-21), a decision has to be made as to whether the paths are real or not. An example of a real path is a flip-flop with a 2x speed clock driving into a flip-flop with a 1x speed clock. An example of a false path is where the designer has explicitly placed clock synchronizer logic between the two clock domains. In this case, even though there appears to be a timing path from one clock domain to the next, it is not a real timing path since the data is not constrained to propagate through the synchronizer logic in one clock cycle. Such a path is referred to as a false path - not real - because the clock synchronizer ensures that the data passes correctly from one domain to the next. False paths between clock domains can be specified using the *set_false_path* specification, such as:

```
set_false_path -from [get_clocks USBCLK] \  
-to [get_clocks MEMCLK]  
# This specification is explained in more detail in Chapter 8.
```

Even though it is not depicted in Figure 2-21, a clock domain crossing can occur both ways, from *USBCLK* clock domain to *MEMCLK* clock domain, and from *MEMCLK* clock domain to *USBCLK* clock domain. Both scenarios need to be understood and handled properly in STA.

What is the reason to discuss paths between clock domains? Typically a design has a large number of clocks and there can be a myriad number of paths between the clock domains. Identifying which clock domain crossings are real and which clock crossings are not real is an important part of the timing verification effort. This enables the designer to focus on validating only the real timing paths.

Figure 2-22 shows another example of clock domains. A multiplexer selects a clock source - it is either one or the other depending on the mode of operation of the design. There is only one clock domain, but two clocks, and these two clocks are said to be mutually-exclusive, as only one clock is active at one time. Thus, in this example, it is important to note that there can never be a path between the two clock domains for *USBCLK* and *USBCLKx2* (assuming that the multiplexer control is static and that such paths do not exist elsewhere in the design).

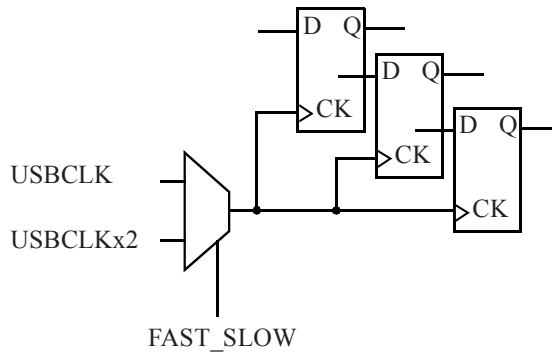


Figure 2-22 *Mutually-exclusive clocks.*

2.10 Operating Conditions

Static timing analysis is typically performed at a specific operating condition¹. An operating condition is defined as a combination of *Process*, *Voltage* and *Temperature* (PVT). Cell delays and interconnect delays are computed based upon the specified operating condition.

There are three kinds of manufacturing process models that are provided by the semiconductor foundry for digital designs: *slow* process models, *typical* process models, and *fast* process models. The *slow* and *fast* process models represent the extreme corners of the manufacturing process of a foundry. For robust design, the design is validated at the extreme corners of the manufacturing process as well as environment extremes for temperature and power supply. Figure 2-23(a) shows how a cell delay changes with the process corners. Figure 2-23(b) shows how cell delays vary with power supply voltage, and Figure 2-23(c) shows how cell delays can vary

1. STA may be performed on a design with cells that have different voltages. We shall see later how these are handled. STA can also be performed statistically, which is described in Chapter 10.

with temperature. Thus it is important to decide the operating conditions that should be used for various static timing analyses.

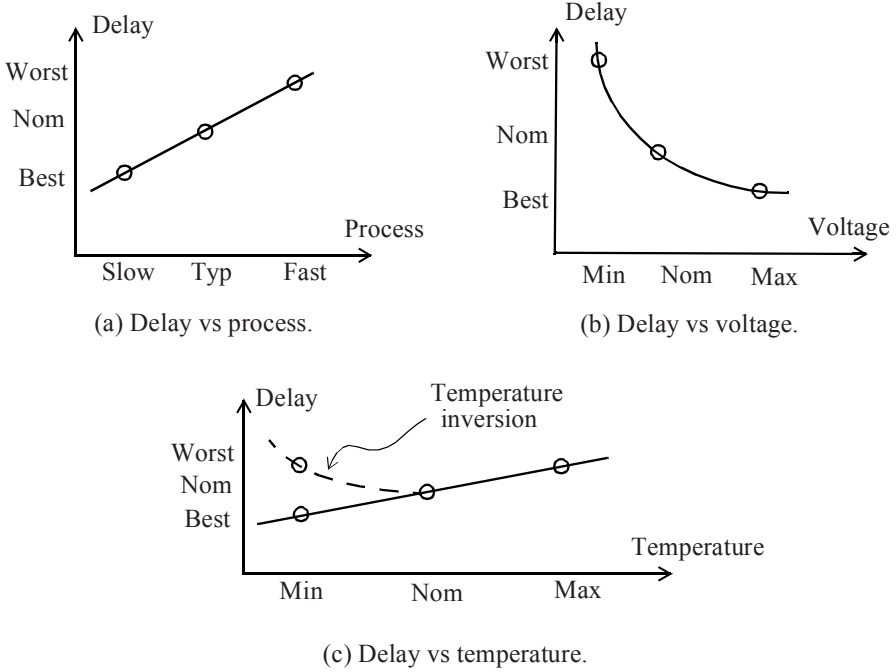


Figure 2-23 Delay variations with PVT.

The choice of what operating condition to use for STA is also governed by the operating conditions under which cell libraries are available. Three standard operating conditions are:

- i. *WCS (Worst-Case Slow)*: Process is *slow*, temperature is highest (say 125C) and voltage is lowest (say nominal 1.2V minus 10%). For nanometer technologies that use low power supplies, there can be another worst-case slow corner that corresponds to the *slow* process, lowest power supply, and lowest temperature. The delays at low temperatures are not always smaller than the de-

lays at higher temperatures. This is because the device threshold voltage (V_t) margin with respect to the power supply is reduced for nanometer technologies. In such cases, at low power supply, the delay of a lightly loaded cell is higher at low temperatures than at high temperatures. This is especially true of high V_t (higher threshold, larger delay) or even standard V_t (regular threshold, lower delay) cells. This anomalous behavior of delays increasing at lower temperatures is called *temperature inversion*. See Figure 2-23(c).

- ii. *TYP (Typical)*: Process is *typical*, temperature is nominal (say 25C) and voltage is nominal (say 1.2V).
- iii. *BCF (Best-Case Fast)*: Process is *fast*, temperature is lowest (say -40C) and voltage is highest (say nominal 1.2V plus 10%).

The environment conditions for power analysis are generally different than the ones used for static timing analysis. For power analysis, the operating conditions may be:

- i. *ML (Maximal Leakage)*: Process is *fast*, temperature is highest (say 125C) and the voltage is also the highest (say 1.2V plus 10%). This corner corresponds to the maximum leakage power. For most designs, this corner also corresponds to the largest active power.
- ii. *TL (Typical Leakage)*: Process is *typical*, temperature is highest (say 125C) and the voltage is nominal (say 1.2V). This refers to the condition where the leakage is representative for most designs since the chip temperature will be higher due to power dissipated in normal operation.

The static timing analysis is based on the libraries that are loaded and linked in for the STA. An operating condition for the design can be explicitly specified using the **set_operating_conditions** command.

```
set_operating_conditions "WCCOM" -library mychip  
# Use the operating condition called WCCOM defined in the  
# cell library mychip.
```

The cell libraries are available at various operating conditions and the operating condition chosen for analysis depends on what has been loaded for the STA.

□